

Relatório - Estrutura de Dados em Java

Questão 1 — Estado da fila após operações (R-4.2)

Operações: enqueue(5), enqueue(3), dequeue(), enqueue(2), enqueue(8), dequeue(), dequeue(), enqueue(9), enqueue(1), dequeue(), enqueue(7), enqueue(6), dequeue(), dequeue(), enqueue(4), enqueue(7), dequeue().

Resultado passo a passo

Operação	Estado da Fila
enqueue(5)	5
enqueue(3)	5 3
dequeue()	3
enqueue(2)	3 2
enqueue(8)	3 2 8
dequeue()	2 8
dequeue()	8
enqueue(9)	8 9
enqueue(1)	8 9 1
dequeue()	9 1
enqueue(7)	9 1 7
enqueue(6)	9 1 7 6
dequeue()	1 7 6
dequeue()	7 6
enqueue(4)	7 6 4
enqueue(7)	7 6 4 7
dequeue()	6 4 7

Nota: a fila segue o princípio FIFO — o primeiro a entrar é o primeiro a sair. O **início** aponta para quem sai e o **fim** para onde entra.

Resposta final: 6 4 7

Questão 2 — TAD Fila com Array (incremento e duplicação)

Descrição

Implementação da Fila usando array circular dinâmico com suporte a duas estratégias de crescimento:

- Incremento fixo: capacidade += crescimento
- Duplicação: capacidade *= 2

Inclui a operação `acessarMenor()` em $O(1)$.

Interface Fila.java

```
package Fila;

public interface Fila{
    public abstract void enqueue(Object o);
    public abstract int size();
    public abstract boolean isEmpty();
    public abstract Object dequeue() throws FilaVaziaExcecao;
    public abstract Object acessarMenor() throws FilaVaziaExcecao;
}
```

Exceção FilaVaziaExcecao.java

```
package Fila;

public class FilaVaziaExcecao extends RuntimeException{
    public FilaVaziaExcecao (String err){
        super(err);
    }
}
```

Implementação FilaArray.java

```
package Fila;

public class FilaArray implements Fila{
    private int capacidade;
    private int crescimento;
    private Object[] arr;
    private int inicio;
    private int fim;
    private int menor;

    public FilaArray(int capacidade, int crescimento){
        this.inicio = 0;
        this.fim = 0;
        this.capacidade = capacidade;
        this.crescimento = crescimento;
        arr = new Object[capacidade];
    }
}
```

```

public void enqueue(Object o){
    if (size() == capacidade - 1){
        grow();
    }

    int valorElemento = (int)o; //precisava converter temporariamente para usar a compar

    if (isEmpty() || menor > valorElemento){
        menor = valorElemento;
    }

    arr[fim] = o;
    fim = (fim + 1) % capacidade;
}

public Object dequeue() throws FilaVaziaExcecao{
    if (isEmpty()){
        throw new FilaVaziaExcecao("A Fila está vazia");
    }

    Object pop_retirado = arr[inicio];
    inicio = (inicio + 1) % capacidade;

    if (!isEmpty()){
        menor = (int)arr[inicio]; //precisa reiniciar o menor
        for (int i = inicio; i != fim; i = (i + 1) % capacidade){
            if (menor > (int)arr[i]){
                menor = (int)arr[i];
            }
        } //foi um array circular
    }

    return pop_retirado;
}

public int size(){
    return (capacidade - inicio + fim) % capacidade;
}

public boolean isEmpty(){
    return fim == inicio; //está vazio
}

public void grow(){
    int novo_capacidade;

```

```

        if (crescimento == 0){
            novo_capacidade = capacidade * 2;
        }
        else{
            novo_capacidade = capacidade + crescimento;
        }

        Object[] novo_arr = new Object[novo_capacidade];
        int novo_inicio = inicio; //temporário que usa

        for (int i = 0; i < size(); i++){
            novo_arr[i] = arr[novo_inicio];
            novo_inicio = (novo_inicio + 1) % capacidade;
        }

        fim = size();
        inicio = 0;
        capacidade = novo_capacidade;
        arr = novo_arr;
    }

    public Object acessarMenor() throws FilaVaziaExcecao{
        if (isEmpty()){
            throw new FilaVaziaExcecao("Não tem elemento na fila!");
        }

        return menor;
    }
}

```

Teste de desempenho da Pilha com Array

O teste foi realizado com a **PilhaArray** inserindo até 1 milhão de elementos via **push**, medindo o tempo em milissegundos para cada combinação de quantidade e estratégia de crescimento.

Quantidade de Elementos	Incremento 10	Incremento 100	Incremento 1000	Duplicação
10	0ms	0ms	0ms	0ms
100	7ms	5ms	6ms	16ms
1000	106ms	90ms	64ms	84ms
10000	351ms	361ms	347ms	590ms
100000	1961ms	2170ms	2045ms	2426ms
1000000	135288ms	130312ms	158007ms	163707ms

Análise do desempenho

Por mais que tenha linha de código `println` para imprimir os números sendo inseridos, percebemos que quando aumentamos o crescimento/incremento, ficava mais rápido o desempenho na maior parte. A duplicação não teve muito bom desempenho.

Questão 3 — TAD Fila e Pilha com Lista Encadeada

Pilha com Lista Encadeada

Interface PilhaLista.java

```
package Fila.Lista.Pilha_Lista;

public interface PilhaLista {
    public void push(Object elemento);
    public Object pop() throws PilhaListaVazia;
    public boolean isEmpty();
    public int size();
}
```

Exceção PilhaListaVazia.java

```
package Fila.Lista.Pilha_Lista;

public class PilhaListaVazia extends RuntimeException{
    public PilhaListaVazia (String err){
        super(err);
    }
}
```

Implementação Pilha_lista.java

```
package Fila.Lista.Pilha_Lista;

public class Pilha_lista implements PilhaLista{
    private class No{
        private Object elemento; //lista
        private No proximo;

        public Object getElemento(){
            return elemento;
        }

        public void setElemento(Object o){
```

```

        elemento = o;
    }
}

private int size;
private No top; //por ser pilha para referencia do topo

public void push(Object elemento){
    No novo = new No();
    novo.setElemento(elemento); //reserva
    novo.proximo = top; //ve o topo
    top = novo; //o topo se tornara o no que está reservado para adicionar
    size++;
}

public Object pop() throws PilhaListaVazia{
    if (isEmpty()){
        throw new PilhaListaVazia("A Pilha está vazia!");
    }

    Object ultimo_elemento = top.getElemento(); //pega o valor ultimo e salvar
    top = top.proximo; //ve o proximo No para andar
    size--;
    return ultimo_elemento;
}

public boolean isEmpty(){
    return top == null;
}

public int size(){
    return size;
}
}

```

Fila com Lista Encadeada

Interface FilaLista.java

```

package Fila.Lista.Fila_Lista;

public interface FilaLista {
    public void enqueue(Object elemento);
    public Object dequeue() throws FilaListaVazia;
    public boolean isEmpty();
    public int size();
}

```

```
}
```

Exceção FilaListaVazia.java

```
package Fila.Lista.Fila_Lista;
```

```
public class FilaListaVazia extends RuntimeException {  
    public FilaListaVazia (String err){  
        super(err);  
    }  
}
```

Implementação Fila_lista.java

```
package Fila.Lista.Fila_Lista;
```

```
public class Fila_lista implements FilaLista{  
    private class No{  
        private Object elemento;  
        private No proximo;  
  
        public Object getElement(){  
            return elemento;  
        }  
  
        public void setElement(Object o){  
            elemento = o;  
        }  
    }  
  
    private int size;  
    private No inicio;  
    private No fim;  
  
    public void enqueue(Object elemento){  
        No novo = new No();  
        novo.setElement(elemento);  
        if (isEmpty()){  
            inicio = novo;  
        } else{  
            fim.proximo = novo;  
        }  
        fim = novo;  
        size++;  
    }  
}
```

```

public Object dequeue() throws FilaListaVazia{
    if (isEmpty()){
        throw new FilaListaVazia("Fila está vazia!");
    }

    Object valor = inicio.getElement();
    inicio = inicio.proximo;
    size--;
    return valor;
}

public boolean isEmpty(){
    return inicio == null;
}

public int size(){
    return size;
}
}

```

Questão 4 — Inversão de Fila em tempo linear

Descrição

Algoritmo que inverte uma fila usando uma pilha auxiliar. O tempo de execução é $O(n)$ — linear.

Implementação InversoTeste.java

```

package Fila.Lista;

import Fila.Lista.Fila_Lista.*;
import Fila.Lista.Pilha_Lista.*;

public class InversoTeste {
    public static void main(String[] args){
        Fila_lista check_fila = new Fila_lista();
        check_fila.enqueue(1);
        check_fila.enqueue(2);
        check_fila.enqueue(3);

        //inverter - a pilha é o responsável para inverter!
        Pilha_lista check_pilha = new Pilha_lista();

        while (!check_fila.isEmpty()){

```



```

        check_pilha.push(check_fila.dequeue());
    } //esvazia a fila para inverter na pilha

    while (!check_pilha.isEmpty()){
        check_fila.enqueue(check_pilha.pop());
    } //esvazia a pilha que já está invertida e coloca na fila a mesma forma

    while (!check_fila.isEmpty()){
        System.out.println(check_fila.dequeue());
    } //imprime a fila e vai esvaziando
}
}

```

Por que é $O(n)$?

Cada operação de enqueue, dequeue, push e pop é $O(1)$. O algoritmo percorre os n elementos duas vezes — total de $2n = O(n)$.

Questão 5 — Fila com Vector

Descrição

Implementação da Fila usando Vector do Java. Mais simples que o array circular — sem necessidade de controlar índices início e fim.

Interface FilaVector.java

```

package Fila.Vector;

public interface FilaVector {
    public void enqueue(Object o);
    public Object dequeue() throws FilaVectorVazia;
    public int size();
    public boolean isEmpty();
}

```

Exceção FilaVectorVazia.java

```

package Fila.Vector;

public class FilaVectorVazia extends RuntimeException {
    public FilaVectorVazia(String err){
        super(err);
    }
}

```

Implementação FilaVectorArray.java

```
package Fila.Vector;

import java.util.Vector;

public class FilaVectorArray implements FilaVector{
    private Vector<Object> lista = new Vector<>();

    public void enqueue(Object o){
        lista.add(o);
    }

    public Object dequeue() throws FilaVectorVazia{
        if (isEmpty()){
            throw new FilaVectorVazia("Fila está vazia");
        }

        return lista.remove(0); //remover do inicio
    }

    public int size(){
        return lista.size();
    }

    public boolean isEmpty(){
        return lista.isEmpty();
    }
}
```

Questão 6 — Sistema de Fila para Clínica

Descrição

Aplicação prática da Fila para gerenciar atendimento de pacientes em uma clínica. Usa `Vector<String>` para armazenar nomes. Possui menu interativo.

Interface FilaClinica.java

```
package Fila.Clinica;

public interface FilaClinica {
    public void enqueue(String nome);
    public String dequeue() throws FilaClinicaVazia;
    public int size();
}
```

```

    public boolean isEmpty();
    //Object só funciona em números, então deve trocar para String
}

```

Exceção FilaClinicaVazia.java

```

package Fila.Clinica;

public class FilaClinicaVazia extends RuntimeException {
    public FilaClinicaVazia (String err){
        super(err);
    }
}

```

Implementação FilaClinicaArray.java

```

package Fila.Clinica;

import java.util.Vector;

public class FilaClinicaArray implements FilaClinica{
    private Vector <String> lista_nomes = new Vector<>();

    public void enqueue(String nome){
        lista_nomes.add(nome);
    }

    public String dequeue() throws FilaClinicaVazia{
        if (isEmpty()){
            throw new FilaClinicaVazia("Não tem nenhum paciente na fila");
        }

        return lista_nomes.remove(0);
    }

    public boolean isEmpty(){
        return lista_nomes.isEmpty(); //se há pacientes
    }

    public int size(){
        return lista_nomes.size(); //quantidade total de pacientes
    }
}

```

Teste TesteFilaClinica.java

```
package Fila.Clinica;

import java.util.Scanner;

public class TesteFilaClinica {
    public static void main(String[] args) throws FilaClinicaVazia{
        FilaClinicaArray check = new FilaClinicaArray();
        int opcao;
        Scanner scanner = new Scanner(System.in); //lê o teclado
        do {
            System.out.println("1 - Nome de um novo paciente");
            System.out.println("2 - Próximo paciente que vai ser atendido");
            System.out.println("3 - Quantidade de pacientes na espera");
            System.out.println("4 - FIM");
            opcao = scanner.nextInt(); //espera a digitação
            System.out.println("Escolha: " + opcao);
            switch (opcao) {
                case 1:
                    scanner.nextLine(); //tira o numero no terminal
                    String nome = scanner.nextLine();
                    check.enqueue(nome);
                    System.out.println("Paciente " + nome + " adicionado!");
                    break;
                case 2:
                    System.out.println("Próximo é: " + check.dequeue());
                    break;
                case 3:
                    System.out.println("Pacientes na espera: " + check.size());
                    break;
            }
        } while (opcao != 4); //porque vai ler depois
    }
}
```

Questão 7 — TAD Fila usando duas Pilhas

Cada pilha tem uma função separada: a primeira pilha é responsável pelo `enqueue`, que realiza o `push` dos elementos, e a segunda pilha é responsável pelo `dequeue`, que realiza o `pop`.

Porém, antes de unir as duas pilhas, é necessário ajustar a ordem dos elementos, pois ao entrar pela primeira pilha via `push` os elementos ficam invertidos e não estariam na ordem correta para uma fila. Por isso, é necessário transferir os

elementos para a segunda pilha, corrigindo a ordem.

Com isso, o tempo de desempenho do **dequeue** é $O(n)$, pois precisa transferir todos os elementos, enquanto o **enqueue** é $O(1)$, pois apenas insere o elemento na pilha.

Operação	Tempo
enqueue	$O(1)$
dequeue	$O(n)$

Questão 8 — TAD Pilha usando duas Filas

A ideia é semelhante à questão anterior, porém invertida. Cada fila tem uma função separada: a primeira fila é responsável pelo **push** e a segunda pelo **pop**.

A diferença é que a fila não inverte a ordem dos elementos como a pilha faz. Por isso, para simular o comportamento LIFO da pilha, é necessário transferir os elementos entre as duas filas para ajustar a ordem, fazendo com que o último elemento inserido seja o primeiro a sair.

Com isso, tanto o **push** quanto o **pop** ficam com tempo de desempenho $O(n)$, pois ambos precisam transferir todos os elementos entre as filas para manter a ordem correta.

Operação	Tempo
push	$O(n)$
pop	$O(n)$

Questão 9 — Estado do Deque após operações (R-4.3)

Operações: insertFirst(3), insertLast(8), insertLast(9), insertFirst(5), removeFirst(), removeLast(), first(), insertLast(7), removeFirst(), last(), removeLast().

Resultado passo a passo

Operação	Estado do Deque
insertFirst(3)	3
insertLast(8)	3 8
insertLast(9)	3 8 9
insertFirst(5)	5 3 8 9

Operação	Estado do Deque
removeFirst()	3 8 9
removeLast()	3 8
first()	3 8 (só espia, retorna 3, não remove)
insertLast(7)	3 8 7
removeFirst()	8 7
last()	8 7 (só espia, retorna 7, não remove)
removeLast()	8

Resposta final: 8

Questão 10 — Deque com Array

Descrição

Implementação do Deque usando array circular. Opera nos dois lados — `insertFirst/removeFirst` pelo início e `insertLast/removeLast` pelo fim. Inclui a operação `acessarMenor()`.

Interface Deque.java

```
package Fila.Deque;

public interface Deque {
    void insertFirst(Object o);
    void insertLast(Object o);
    Object removeFirst() throws DequeVazia;
    Object removeLast() throws DequeVazia;
    Object first() throws DequeVazia;
    Object last() throws DequeVazia;
    int size();
    boolean isEmpty();
    Object acessarMenor() throws DequeVazia;
}
```

Exceção DequeVazia.java

```
package Fila.Deque;

public class DequeVazia extends RuntimeException{
    public DequeVazia(String err){
        super(err);
    }
}
```

Implementação DequeArray.java

```
package Fila.Deque;

public class DequeArray implements Deque {

    private Object[] arr;
    private int capacidade, inicio, fim, size, menor;

    public DequeArray(int capacidade){
        this.capacidade = capacidade;
        this.inicio = 0;
        this.fim = 0;
        arr = new Object[capacidade];
    }

    public void insertFirst(Object o){
        inicio = (inicio - 1 + capacidade) % capacidade;
        arr[inicio] = o; //coloca o valor novo

        int elemento = (int)o;
        if (isEmpty() || menor > elemento){
            menor = elemento;
        }

        size++;
    }

    public void insertLast(Object o){
        arr[fim] = o;
        fim = (fim + 1) % capacidade; //a ordem importa e precisa ficar circular para nao f

        int elemento = (int)o;
        if (size == 0 || menor > elemento){
            menor = elemento;
        }

        size++;
    }

    public Object removeFirst() throws DequeVazia{
        if (isEmpty()){
            throw new DequeVazia("Deque está vazia!");
        }

        Object valor = arr[inicio];
```

```

        inicio = (inicio + 1) % capacidade;

        size--;

        if (!isEmpty()){
            menor = (int)arr[inicio];
            for (int i = inicio; i != fim; i = (i + 1) % capacidade){
                if (menor > (int)arr[i]){
                    menor = (int)arr[i];
                }
            }
        }

        return valor;
    }

    public Object removeLast() throws DequeVazia{
        if (isEmpty()){
            throw new DequeVazia("Deque está vazia!");
        }

        fim = (fim - 1 + capacidade) % capacidade;
        Object valor = arr[fim];
        size--;

        if (!isEmpty()){
            menor = (int)arr[inicio];
            for (int i = inicio; i != fim; i = (i + 1) % capacidade){
                if (menor > (int)arr[i]){
                    menor = (int)arr[i];
                }
            }
        }

        return valor;
    }

    public Object first() throws DequeVazia{
        if (isEmpty()){
            throw new DequeVazia("Deque está vazia!");
        }

        return arr[inicio]; //quero o elemento, por isso está dentro de uma lista
    }

    public Object last() throws DequeVazia{

```



```

        if (isEmpty()){
            throw new DequeVazia("Deque está vazia!");
        }

        return arr[(fim - 1 + capacidade) % capacidade];
        //está de forma circular e o fim pode estar qualquer canto do array e passar, entao
    }

    public boolean isEmpty(){
        return size == 0;
    }

    public int size(){
        return size;
    }

    public Object acessarMenor() throws DequeVazia{
        if (isEmpty()) {
            throw new DequeVazia("Deque está sem elementos");
        }

        return menor;
    }
}

```

Tabela de Tempos de Execução

As operações de inserção e consulta do Deque com array têm tempo **O(1)**, pois acessam diretamente pelo índice. As operações de remoção têm tempo **O(n)** pois recalculam o menor elemento percorrendo o array.

Operação	Tempo	Motivo
insertFirst	O(1)	acessa direto pelo índice inicio
insertLast	O(1)	acessa direto pelo índice fim
removeFirst	O(n)	recalcula o menor percorrendo o array
removeLast	O(n)	recalcula o menor percorrendo o array
first	O(1)	acessa direto pelo índice inicio
last	O(1)	acessa direto pelo índice fim
isEmpty	O(1)	compara size direto
size	O(1)	retorna atributo direto
acessarMenor	O(1)	retorna atributo direto

Questão 11 — Deque com Vector

Descrição

Implementação do Deque usando Vector do Java. Mais simples que o array circular — sem necessidade de controlar índices.

Interface DequeVector.java

```
package Fila.Deque.DequeVector;

public interface DequeVector {
    void insertFirst(Object o);
    void insertLast(Object o);
    Object removeFirst() throws DequeVectorVazia;
    Object removeLast() throws DequeVectorVazia;
    Object last() throws DequeVectorVazia;
    Object first() throws DequeVectorVazia;
    int size();
    boolean isEmpty();
}
```

Exceção DequeVectorVazia.java

```
package Fila.Deque.DequeVector;

public class DequeVectorVazia extends RuntimeException {
    public DequeVectorVazia (String err){
        super(err);
    }
}
```

Implementação DequeVectorArray.java

```
package Fila.Deque.DequeVector;

import java.util.Vector;

public class DequeVectorArray implements DequeVector{
    private Vector <Object> lista = new Vector<>();

    public void insertFirst(Object o){
        lista.insertElementAt(o, 0);
    }

    public void insertLast(Object o){
        lista.add(o); //ele já adiciona no fim
    }
}
```

```

    }

    public Object removeFirst() throws DequeVectorVazia{
        if (isEmpty()){
            throw new DequeVectorVazia("Deque está vazia!");
        }

        return lista.remove(0);
    }

    public Object removeLast() throws DequeVectorVazia{
        if (isEmpty()){
            throw new DequeVectorVazia("Deque está vazia!");
        }

        return lista.remove(lista.size() - 1);
    }

    public Object first() throws DequeVectorVazia{
        if (isEmpty()){
            throw new DequeVectorVazia("Deque está vazia!");
        }

        return lista.firstElement();
    }

    public Object last() throws DequeVectorVazia{
        if (isEmpty()){
            throw new DequeVectorVazia("Deque está vazia!");
        }

        return lista.lastElement();
    }

    public int size(){
        return lista.size();
    }

    public boolean isEmpty(){
        return lista.isEmpty();
    }
}

```

Tabela de Tempos de Execução

Operação	Tempo	Motivo
<code>insertFirst</code>	$O(n)$	<code>insertElementAt(0)</code> desloca todos os elementos
<code>insertLast</code>	$O(1)$	<code>add</code> no fim do Vector
<code>removeFirst</code>	$O(n)$	<code>remove(0)</code> desloca todos os elementos
<code>removeLast</code>	$O(1)$	<code>remove(size-1)</code> acessa direto
<code>first</code>	$O(1)$	<code>firstElement()</code> acessa direto
<code>last</code>	$O(1)$	<code>lastElement()</code> acessa direto
<code>isEmpty</code>	$O(1)$	verifica direto
<code>size</code>	$O(1)$	retorna tamanho direto

Questão 12 — Operação AcessarMenor em $O(1)$

Descrição

Modificação dos TADs Pilha, Fila e Deque implementados com arrays para incluir a operação `acessarMenor()` que retorna o menor valor armazenado com complexidade $O(1)$.

A estratégia é manter um atributo `menor` sempre atualizado a cada inserção e remoção, evitando percorrer o array toda vez que `acessarMenor()` for chamado.

Tabela de Tempos — Fila com Array

Operação	Tempo	Motivo
<code>enqueue</code>	$O(1)$	insere direto pelo índice <code>fim</code>
<code>dequeue</code>	$O(n)$	recalcula o menor percorrendo o array
<code>isEmpty</code>	$O(1)$	compara <code>inicio</code> e <code>fim</code> direto
<code>size</code>	$O(1)$	cálculo direto com índices
<code>acessarMenor</code>	$O(1)$	retorna atributo direto
<code>grow</code>	$O(n)$	copia todos os elementos para novo array

Tabela de Tempos — Pilha com Array

Operação	Tempo	Motivo
<code>push</code>	$O(1)$	insere direto pelo índice <code>top</code>

Operação	Tempo	Motivo
pop	$O(n)$	recalcula o menor percorrendo o array
top	$O(1)$	acessa direto pelo índice <code>top</code>
isEmpty	$O(1)$	compara <code>top</code> com -1 direto
size	$O(1)$	retorna <code>top + 1</code> direto
empty	$O(1)$	reseta <code>top</code> para -1 direto
acessarMenor	$O(1)$	retorna atributo direto

Tabela de Tempos — Deque com Array

Operação	Tempo	Motivo
insertFirst	$O(1)$	acessa direto pelo índice <code>inicio</code>
insertLast	$O(1)$	acessa direto pelo índice <code>fim</code>
removeFirst	$O(n)$	recalcula o menor percorrendo o array
removeLast	$O(n)$	recalcula o menor percorrendo o array
first	$O(1)$	acessa direto pelo índice <code>inicio</code>
last	$O(1)$	acessa direto pelo índice <code>fim</code>
isEmpty	$O(1)$	compara <code>size</code> direto
size	$O(1)$	retorna atributo direto
acessarMenor	$O(1)$	retorna atributo direto